Dynamic Programming & String Algorithms

Assignment 4 · Beibarys Balykbayev · SE-2516 · 8 November 2026

Experiment Design Card

Written and committed before the first benchmark; the original is EXPERIMENT_DESIGN.md. Plan revisions: none.

Questions	**Edit:** how do recursion, a full table, and two rows change work, time, and storage as input length grows? **Search:** how does repeated near-matching text change Naive and KMP comparison counts and time?
Sizes	All edit versions: n = 3, 6, 9, 12. DP extension: 128, 256, 512, 1024, 2048. Both strings have length n. Search: text n = 4096, 16384, 65536, 262144; pattern m = n/64. Every series has at least four sizes and spans at least 4x. Small edit cases bound recursion; large cases give useful times.
Inputs	Edit: **identical** generated strings exercise matching shortcuts; **disjoint** alphabets force three recursive branches at every non-base call (hard case). Search: **random** text/pattern over 26 letters exposes quick mismatches; **near_match** uses A^n versus A^(m-1)B to repeat long failed checks (hard case). Search cases isolate comparisons with no matches; tests cover successful and overlapping matches.
Metrics	Median of five timed runs after three untimed warmups per case. Edit: non-base state evaluations, calls, character comparisons, peak live int-array cells, and recursion depth. Search: character comparisons separately in search and LPS, plus their sum. Counters are incremented inside the algorithms; storage counts exclude object overhead and call frames.
Prediction	Disjoint recursion grows exponentially; identical recursion needs n+1 calls. Both DP versions evaluate n^2 cells; table storage is quadratic and two-row storage linear. Naive is near-linear on random input but approaches n*m on near matches. Since m grows with n, that series should be quadratic; KMP including LPS remains linear.
Plots	Edit work versus n (log y) tests repeated work. DP storage (log-log) tests quadratic versus linear space. Search comparisons (log-log) contrasts input types. Median runtime panels show practical costs. Each chart includes units and a one-sentence takeaway.
Reproduction	Own Java generator: fixed seed 20261108, fresh case-specific seed derived from family/type/n/m. Save SHA-256 input fingerprints. Same commands reproduce inputs, answers, and counters; clock readings vary. Generate and validate outside timing. Stop recursion at n=12 by design, without claiming an absolute machine limit.

Measurement protocol

The CSV contains 300 individual measured runs across 60 algorithm/input/size cases. Timing uses System.nanoTime and includes counter updates, algorithm allocation, and KMP preprocessing. Cases run sequentially in a fixed order, without forced garbage collection. JIT compilation and cache effects can remain after three warmups.

State, transition, base case

D(i,j) is the edit distance between prefixes of lengths i and j. Equal last characters use D(i-1,j-1); otherwise use 1 + min(D(i-1,j), D(i,j-1), D(i-1,j-1)). The base cases are D(0,j)=j and D(i,0)=i. The same recurrence is documented above each implementation.

Edit Distance: work and storage

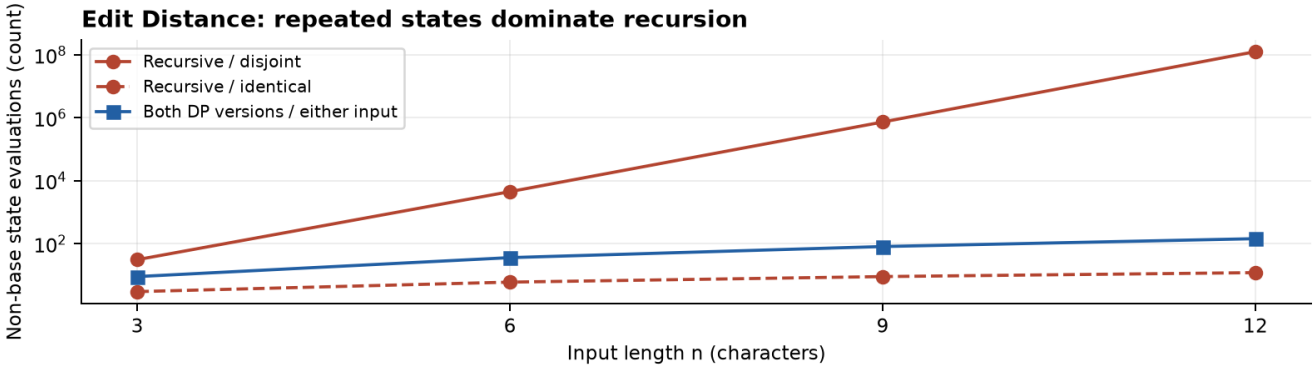


Figure 1. At n=12, disjoint recursion evaluates 125,797,984 non-base states; each DP version evaluates only 144.

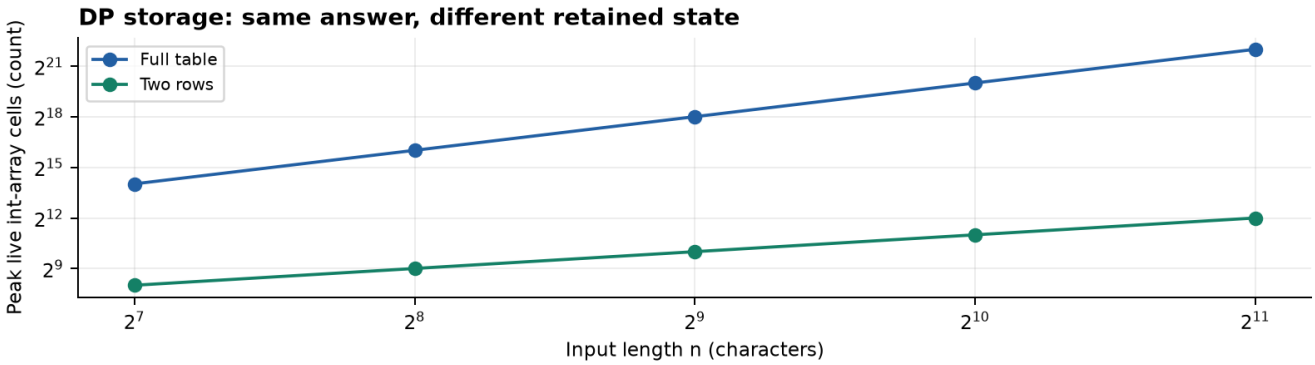


Figure 2. At n=2048, two rows retain 4,098 integer cells versus 4,198,401 for the table (1024.5x fewer).

Prediction versus result

The work and storage predictions match the counters. Disjoint recursion rises from 2,193,844 calls at n=9 to 377,393,953 at n=12. Identical recursion at n=12 takes 13 calls because every comparison follows one diagonal branch. Both DP versions still evaluate all n^2 non-base cells for either input type; they do not skip the table on identical inputs.

Case	Median (ms)	Measured work / storage
Recursive, disjoint n=12	1132.992	377,393,953 calls; depth 24
Full table, disjoint n=2048	18.728	4,194,304 states; 4,198,401 cells
Two rows, disjoint n=2048	15.756	4,194,304 states; 4,098 cells

Storage is a count of retained int-array elements, observed at allocation sites. Multiplying by four estimates only the integer payload, not Java heap usage. The recursive version retains no integer arrays but does use a call stack; its recorded depth is separate.

String search: structure matters

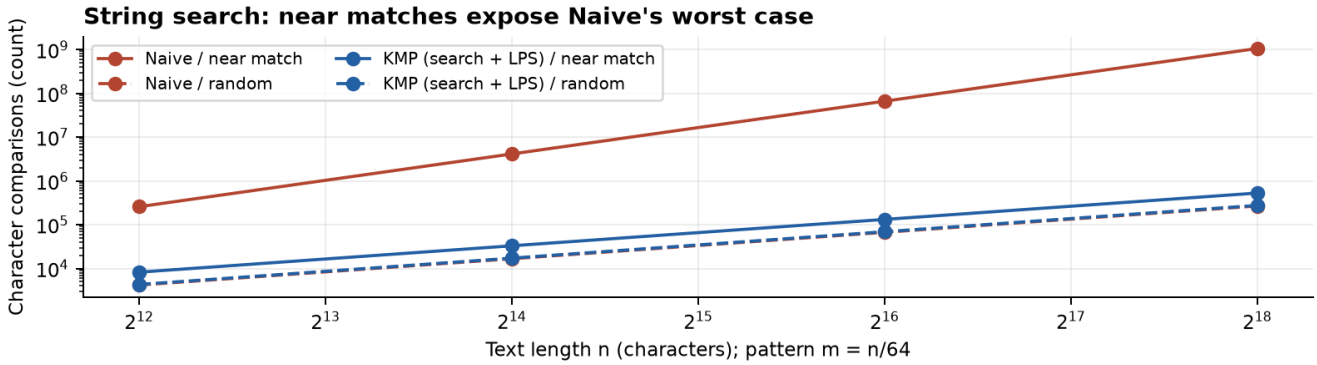


Figure 3. On the largest near-match case, Naive uses 2,000x as many comparisons as KMP including LPS construction.

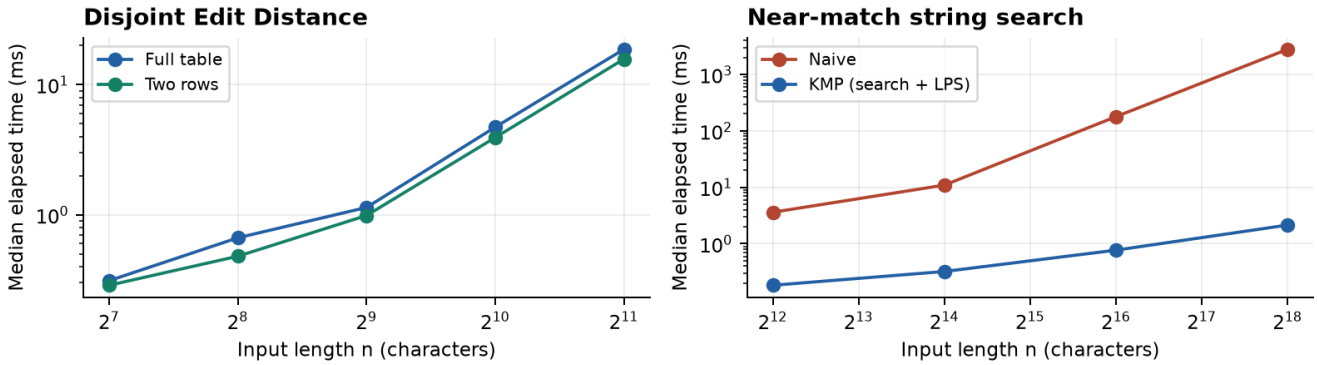


Figure 4. Median time separates the expensive near-match Naive scans, while the DP timing curves show overhead as well as increasing work.

Prediction versus result

The search prediction matches measured work. At n=262144 and m=4096, Naive performs 1,056,968,704 comparisons, exactly m(n-m+1); KMP performs 520,193 search comparisons plus 8,189 LPS comparisons, totaling 528,382. Increasing n fourfold also increases m fourfold, so Naive's near-match count grows about sixteenfold, while KMP's count grows about fourfold.

For the largest random case, Naive and KMP use 268,269 and 276,338 comparisons. Quick first-character mismatches keep both close to linear, and KMP preprocessing adds a small cost. All generated benchmark search cases have zero matches; separate reference and random tests verify the lists of actual matches, including overlaps.

At the largest near-match size, measured medians are 2791.404 ms (Naive) and 2.145 ms (KMP). These times describe this JVM run; the comparison counts provide the stronger evidence about growth. Naive's time on the smallest random case is higher than on the next size, consistent with residual JIT and other runtime overhead rather than fewer comparisons at larger n.

Interpretation and short answers

Which input made string search slowest, and why?

The slowest measured search was Naive on near_match at n=262144 and m=4096, with a 2791.404 ms median. Every candidate start agrees with the first m-1 pattern characters and fails on the final B. Naive repeats those checks at all n-m+1 starts, giving m(n-m+1) comparisons. KMP reuses the known border through LPS and keeps its text index moving forward.

How large an n could recursive Edit Distance handle?

The largest tested equal length was n=12; disjoint input completed with a 1132.992 ms median. It made 377,393,953 recursive calls, compared with 2,193,844 at n=9. The experiment stopped at the planned cap because repeated subproblems grow exponentially, not because a timeout or stack failure occurred. Matching inputs are much easier, so n=12 is a tested limit for this experiment rather than the largest possible input on this machine.

What do two rows gain and lose compared with the table?

Two rows keep only the previous and current rows, reducing extra storage from O(n*m) to O(m) while retaining O(n*m) time and the exact distance. At n=m=2048, the measured retained integer cells drop from 4,198,401 to 4,098. Discarded rows prevent straightforward backtracking through the full table to reconstruct the edit sequence. Recovering that sequence needs additional storage, recomputation, or a different reconstruction method.

Which string search would suit the city platform?

I would choose KMP for finding literal patterns in long city-platform logs because it has O(n+m) time even on repetitive input. Its O(m) LPS array avoids the repeated prefix checks that make Naive slow in the adversarial experiment. Naive is still easy to implement and can be competitive for short, mostly mismatching inputs. Street-name spell checking needs Edit Distance or another approximate matching method, because KMP alone cannot correct typos.

What the measured times mean

Times use five independent invocations of each deterministic case and report their median, after three warmups. Small cases remain sensitive to JIT compilation, timer overhead, CPU scheduling, caches, and allocation or garbage collection; fractional milliseconds are retained instead of rounding to zero. The JVM ran with a 256 MiB initial heap and a 1 GiB maximum heap; the exact runtime and measurement timestamp are recorded in results/environment.txt. Instrumentation is included in timing, so these are timings of the counted implementations. The state, comparison, and storage counters support growth conclusions when clock ratios are noisy.

Validation and reproducibility

All 17 JUnit 5 test invocations pass: reference distances and LPS arrays, empty inputs, overlaps, 100 random short edit pairs, 100 random search texts against String.indexOf, 100 brute-force LPS checks, and counter/reset checks. The analysis script verifies all 300 CSV rows, five runs per case, consistent fingerprints/counters, and expected case coverage. Production code has no external libraries. Run scripts/reproduce.sh to rebuild the experiments, plots, and PDF.

Repository: github.com/lackoftalentt/daa-assigment4 · branch main · release tag v1.0.
AI acknowledgement: OpenAI Codex assisted with implementation, tests, experiments, analysis, and writing; the saved results were measured locally.